Частное образовательное учреждение высшего образования
«Санкт-Петербургский институт экономики и управления»
(СПбИЭУ)

Специальность 09.02.07 «Информационные системы и программирование»
ПМ.03 Сопровождение и обслуживание программного обеспечения компьютерных систем
МДК.03.01 Внедрение и поддержка компьютерных систем

ОТЧЁТ

по промежуточному контролю № 15

Тема 1.1. Основные методы внедрения и анализа функционирования ПО
«Проектирование релизного процесса, Runbook и ADR для IT-проекта»
Вариант 10

Выполнил: студент группы И14-23-9
Иванов Егор Михайлович

Проверила: преподаватель
Чудинова Александра Анатольевна
Оценка: _____

Санкт-Петербург
2026

1. Описание задания

Цель практикума: на практике отработать элементы CALS-концепции — составить регламент окна обслуживания, описать план отката (Rollback), написать аварийный Runbook и составить запись об архитектурном решении (ADR) для проекта в среде .NET / Visual Studio.

Исходные условия (сценарий). Студент — ведущий разработчик (SRE-инженер) проекта **OrderService** на C# / .NET. На Production выкатывается критическое обновление — переход со старой версии платёжного шлюза на новую. Документы оформляются как Markdown-документация в папке `/docs` проекта Visual Studio.

Индивидуальный вариант 10 — кейс 10 «Внедрение защищённого WAF (Web Application Firewall) перед API» (категория 3 «Инфраструктура, масштабирование и безопасность»):

– **Ситуация:** на фронтенде развёртывается система защиты от DDoS-атак и SQL-инъекций на уровне прокси-сервера.

– **Специфика:** риск того, что легитимные сложные JSON-запросы от мобильного приложения будут ошибочно заблокированы правилами WAF.

Требуется: (1) регламент — окно обслуживания, smoke-тесты после деплоя (`GET /health` → 200 и `DbContext.Database.CanConnect()`), план отката (условие: доля ошибок 5xx > 3 % в первые 3 минуты; действие: `kubectl rollout undo deployment/order-service`, до 30 секунд); (2) мини-Runbook на инцидент «504 Gateway Timeout при обращении к платёжному шлюзу»; (3) ADR `adr-001-payment-gateway.md`; (4) ответ на вопрос, какой из трёх элементов сложнее всего внедрить в команде.

2. Ход выполнения

2.1. Связь сценария и кейса 10

Новый платёжный шлюз принимает трафик только от API, закрытого WAF: для публичных веб-приложений, обрабатывающих платёжные данные, это требование стандарта PCI DSS (требование 6.4). Поэтому релиз OrderService 2.0.0 разделён на **два независимых изменения**, у каждого — свои smoke-тесты и свой откат. Это следует правилу из лекции: «никогда не делай деплой без плана отката».

Таблица 1 – Состав релиза OrderService 2.0.0

№	Изменение	Что меняется	Откат
CH-1	WAF перед API (кейс 10)	Ingress NGINX + ModSecurity + OWASP CRS: SQL-инъекции, XSS, лимит тела 64 КБ, 50 запросов/с с IP	WAF → DetectionOnly (≈ 10 с)
CH-2	Новый платёжный шлюз	IPaymentService → NewGatewayPaymentService (3-D Secure 2.0)	kubectl rollout undo (≤ 30 с) / Payments__Provider=Legacy

Чтобы правила WAF не заблокировали мобильное приложение в первый же день, за 7 дней до окна WAF включается в режиме **DetectionOnly** (только журнал): команда разбирает срабатывания и добавляет исключения, а в окне обслуживания режим меняется на **On**.

2.2. Регламент: окно обслуживания

Таблица 2 – Окно обслуживания (Maintenance Window)

Вопрос	Решение
Когда планируем работы	Среда, 28.10.2026, 02:00–03:00 МСК (60 мин, из них 20 мин — резерв на откат)
Почему это время	минимум заказов ночью (< 2 % суточного трафика); сразу после бэкапа БД 01:30; середина недели — не пятница
Как предупреждаем пользователей	за 24 часа — баннер в личном кабинете и push в мобильном приложении; за 1 час — повторный баннер
Кого ещё предупреждаем	Service Desk (L1), поддержку платёжного провайдера, дежурного архитектора
Заморозка изменений	с 27.10 18:00 до 28.10 12:00 других деплоев нет (Code Freeze)

2.3. Smoke-тесты (дым-тесты) после деплоя

Таблица 3 – Smoke-тесты релиза 2.0.0

№	Действие	Ожидаемый результат
1	Запрос GET /health	статус 200, status = Healthy, version = 2.0.0

2	Проверка подключения к БД методом DbContext.Database.CanConnect()	true
3	POST /api/v1/orders со «сложным» JSON мобильного приложения (вложенные объекты, апострофы, слова select / order by)	201 Created — нет ложного срабатывания WAF
4	Контрольная атака: ?search=1' OR '1'='1, UNION SELECT, <script>	403 Forbidden, в ответе ruleId правила
5	Заголовок X-Waf-Mode ответа	On

Тесты № 1–5 автоматизированы в проекте `tests/OrderService.Tests` (xUnit + WebApplicationFactory) и запускаются командой `dotnet test --filter Category=Smoke` сразу после деплоя.

2.4. План отката (Rollback Plan)

Таблица 4 – План отката

Условие активации отката (любое)	**Техническое действие**	**Время**
Доля ошибок 5xx > 3 % в течение первых 3 минут после переключения трафика	kubectl rollout undo deployment/order-service -n production kubectl rollout status deployment/order-service -n production	до 30 с
Сбой только платёжной части	kubectl set env deployment/order-service Payments__Provider=Legacy -n production	≈ 30 с
(кейс 10) Доля 403 от WAF на запросы мобильного приложения > 0,5 % за 5 минут	kubectl apply -f deploy/k8s/order-service-ingress-waf-detection.yaml (WAF → DetectionOnly)	≈ 10 с
Не пройден любой smoke-тест	откат изменения, к которому относится тест (СН-1 или СН-2)	—

При откате WAF не выключается полностью: режим DetectionOnly сохраняет журнал атак и защиту от DDoS (лимит частоты запросов), но перестаёт блокировать запросы по сигнатурам.

2.5. Документ А. Мини-Runbook для дежурного инженера

Листинг 1 – docs/runbooks/RB-001-payment-gateway-504.md

```
# Runbook RB-001. 504 Gateway Timeout при обращении к платёжному шлюзу

**Сервис:** OrderService 2.0.0 · **Алерт:** `OrderService5xxRate` / рост ответов 504 на
`POST /api/v1/orders/{id}/pay`
**Кому:** дежурный инженер (SRE / Support) · **Время реакции:** 5 минут

## Инцидент

Сервис вернул ошибку **504 Gateway Timeout** при обращении к платёжному шлюзу:
покупатели не могут оплатить заказ, в логах — `504 Gateway Timeout при оплате
заказа ...`.

## Шаг 1. Диагностика

Проверьте доступность внешнего API шлюза:
```

```
```bash
curl -I https://api.payment-gateway.com/v1/ping
```

* Ответ `HTTP/2 200` — внешний шлюз **работает**, проблема на нашей стороне → **Шаг 2**.
* Нет ответа / `5xx` — сбой у провайдера → сообщите в Service Desk и сразу **Шаг 3**.

Шаг 2. Действие

Если внешний шлюз отвечает, но локальный пул соединений исчерпался, перезапустите пул
воркеров:

```bash
systemctl restart order-service-worker
```

Через 1 минуту повторите тестовую оплату служебного заказа и проверьте долю 504 на
дашборде.

Шаг 3. Эскалация

Если **через 2 минуты** статус не изменился — **немедленно** свяжитесь с дежурным
архитектором
по каналу **`#emergency-sre`**. Сообщите: время начала, результат шагов 1–2, TraceId
одного
неуспешного запроса. Решение об откате (`Payments__Provider=Legacy`) принимает
архитектор.
```

Для кейса 10 дополнительно составлен Runbook **RB-002** «WAF блокирует легитимные запросы мобильного приложения (403)»: диагностика по журналу ModSecurity (номер правила `id`), действие — перевод WAF в DetectionOnly, эскалация в `#emergency-sre` через 2 минуты (файл `docs/runbooks/RB-002-waf-false-positive-403.md`).

2.6. Документ Б. ADR (архитектурное решение)

Листинг 2 – docs/adr/adr-001-payment-gateway.md

```
# ADR-001. Подключение нового платёжного шлюза через интерфейс IPaymentService

**Статус:** Утверждено.

**Дата:** 21.10.2026 · **Автор:** Иванов Е.М. · **Связано с:** релиз OrderService 2.0.0,
ADR-002

## Контекст

Требуется подключить новый безопасный платёжный шлюз с поддержкой протокола **3D-Secure
2.0**.
Старый шлюз 3-D Secure 2.0 не поддерживает: банки чаще отклоняют такие платежи, а
покупатель
не получает frictionless-оплату (без ввода кода) на небольшие суммы. Шлюз поставляется
со своим SDK, формат ответов которого отличается от старого.

## Решение

Интегрируем официальный SDK платёжной системы через абстрактный интерфейс
**`IPaymentService`** в C#,
чтобы изолировать бизнес-логику от деталей конкретного вендора.

* `IPaymentService.ChargeAsync(PaymentRequest)` возвращает доменный `PaymentResult`
  (`Approved`, `ChallengeRequired`, `Declined`) — бизнес-логика не видит классов SDK.
* `NewGatewayPaymentService` — адаптер над SDK нового шлюза;
`LegacyGatewayPaymentService` — старый шлюз.
* Реализация выбирается в DI по настройке `Payments:Provider`, поэтому платёжную часть
```

```
можно
  откатить переменной окружения без пересборки.

## Последствия

* **Плюсы:** упрощается написание модульных тестов с использованием **Moq** — шлюз
подменяется
  mock-объектом (`tests/OrderService.Tests/RollbackAndPaymentTests.cs`); смена вендора в
будущем
  затрагивает только адаптер.
* **Минусы:** требуется **строгая валидация контрактов при изменении версии SDK** — при
обновлении
  SDK обязательно прогоняются тесты адаптера и smoke-тест тестовой оплаты, иначе
изменение
  формата ответа вендора незаметно сломает оплату.
```

Решение по кейсу 10 оформлено отдельной записью **ADR-002** «WAF перед API: ModSecurity + OWASP CRS на Ingress NGINX»: JSON-тело разбирается на поля (`ctl:requestBodyProcessor=JSON`), Paranoia Level 1, ввод в два этапа (DetectionOnly → On). Манифесты Ingress для режима On и для отката — в `deploy/k8s/`.

3. Программный код с комментариями

Чтобы регламент, smoke-тесты и план отката можно было проверить, а не только описать, в решении `OrderService.sln` реализован учебный сервис: эндпоинт `/health` с проверкой БД, абстракция платёжного шлюза `IPaymentService` (ADR-001) и стенд WAF `WafStandMiddleware` с режимами On / DetectionOnly / Off — по аналогии с директивой ModSecurity `SecRuleEngine`.

Таблица 5 – Структура решения

Файл	**Назначение**
src/OrderService.Api/Program.cs	API: /health (CanConnect), заказы, оплата, выбор шлюза, WAF в конвейере
src/OrderService.Api/Payments/*.cs	IPaymentService, новый шлюз 3-D Secure 2.0, старый шлюз (откат)
src/OrderService.Api/Waf/ WafStandMiddleware.cs	стенд WAF: сигнатуры SQLi/XSS, лимит тела, проверка значений JSON
tests/OrderService.Tests/*.cs	smoke-тесты релиза, тест отката, тесты оплаты с Moq
docs/, deploy/k8s/, CHANGELOG.md	регламент, Runbook, ADR, манифесты Ingress, журнал изменений

Исходный код решения и документация в репозитории GitHub: https://github.com/Egorvan606-senior/mdk03/tree/main/pk-15

Листинг 3 – Program.cs — регистрация шлюза, WAF и smoke-эндпоинт /health

```
// ---- Хранилище заказов -----------------------------------------------------
```

```
builder.Services.AddDbContext<OrderDbContext>(options =>
    options.UseSqlite(builder.Configuration.GetConnectionString("Orders") ?? "Data
Source=orders.db"));

// ---- Платёжный шлюз (ADR-001): реализация выбирается настройкой --------------
// Payments:Provider = NewGateway3DS2 (релиз 2.0.0) | Legacy (быстрый откат без
пересборки)
if (string.Equals(builder.Configuration["Payments:Provider"], "Legacy",
StringComparison.OrdinalIgnoreCase))
{
    builder.Services.AddScoped<IPaymentService, LegacyGatewayPaymentService>();
}
else
{
    builder.Services.AddScoped<IPaymentService, NewGatewayPaymentService>();
}

var app = builder.Build();

// Схема БД создаётся при старте (для учебного стенда вместо миграций)
using (var scope = app.Services.CreateScope())
{
    scope.ServiceProvider.GetRequiredService<OrderDbContext>().Database.EnsureCreated();
}

// ---- WAF перед API (ADR-002): первым в конвейере, до обработки запроса ------
app.UseMiddleware<WafStandMiddleware>();

// ---- Smoke-тест № 1 и № 2: GET /health (ожидаем 200) + CanConnect к БД --------
app.MapGet("/health", (OrderDbContext db, IPaymentService payments, IConfiguration cfg)
=>
{
    bool dbOk = db.Database.CanConnect();
    var body = new
    {
        status = dbOk ? "Healthy" : "Unhealthy",
        version = "2.0.0",
        database = dbOk ? "Connected" : "Unavailable",
        paymentProvider = payments.ProviderName,
        wafMode = cfg["Waf:Mode"] ?? "On",
        checkedAtUtc = DateTime.UtcNow
    };
    // 503 — сигнал Kubernetes readiness-probe не направлять трафик на этот под
    return dbOk ? Results.Ok(body) : Results.Json(body, statusCode:
StatusCodes.Status503ServiceUnavailable);
});

var orders = app.MapGroup("/api/v1/orders");
```

Листинг 4 – IPaymentService.cs — абстракция платёжного шлюза (ADR-001)

```
namespace OrderService.Payments;

/// <summary>
/// Абстракция платёжного шлюза (решение ADR-001).
/// Бизнес-логика заказов зависит только от этого интерфейса и не знает деталей SDK
/// конкретного вендора, поэтому:
///   * шлюз меняется регистрацией в DI (и откатывается настройкой Payments:Provider);
///   * в модульных тестах интерфейс подменяется mock-объектом (Moq).
/// </summary>
public interface IPaymentService
{
    /// <summary>Название провайдера — выводится в /health для контроля после
деплоя.</summary>
    string ProviderName { get; }

    Task<PaymentResult> ChargeAsync(PaymentRequest request, CancellationToken ct);
}

public sealed record PaymentRequest(Guid OrderId, decimal Amount, string Currency, string
CardToken);

public enum PaymentStatus
{
```

```
    Approved,           // оплата прошла (frictionless-сценарий 3-D Secure 2.0)
    ChallengeRequired,  // банк требует подтверждения покупателем (challenge 3-D Secure
2.0)
    Declined            // отказ банка
}

public sealed record PaymentResult(PaymentStatus Status, string ProviderTransactionId,
string? ChallengeUrl = null);

/// <summary>Шлюз не ответил за отведённое время — API отдаёт 504 Gateway Timeout (см.
Runbook RB-001).</summary>
public sealed class PaymentGatewayTimeoutException(string message) : Exception(message);
```

Листинг 5 – WafStandMiddleware.cs — правила и проверка запроса

```
using System.Diagnostics;
using System.Text.Json;
using System.Text.RegularExpressions;

namespace OrderService.Waf;

/// <summary>Режим работы WAF — по аналогии с директивой ModSecurity SecRuleEngine.</summary>
public enum WafMode
{
    Off,            // правила не применяются
    DetectionOnly,  // срабатывания только журналируются (режим «наблюдения» и режим отката)
    On              // запросы, нарушающие правила, блокируются ответом 403
}

/// <summary>Правило WAF (упрощённый аналог правил OWASP Core Rule Set).</summary>
public sealed record WafRule(string Id, string Description, Regex Pattern);

/// <summary>
/// Учебный стенд WAF перед API (кейс 10). В production эту роль выполняет
/// ModSecurity + OWASP CRS на Ingress NGINX (deploy/k8s/order-service-ingress-waf.yaml);
/// здесь те же принципы реализованы middleware, чтобы smoke-тесты и план отката можно
/// было проверить локально в Visual Studio:
///   * сигнатуры SQL-инъекций (942xxx) и XSS (941xxx);
///   * ограничение размера тела запроса (защита от «тяжёлых» запросов при DDoS);
///   * JSON-тело разбирается и проверяются ЗНАЧЕНИЯ полей, а не сырой текст —
///     это главная мера против ложных срабатываний на сложных JSON мобильного приложения
///     (в ModSecurity — ctl:requestBodyProcessor=JSON).
/// </summary>
public sealed class WafStandMiddleware(RequestDelegate next, IConfiguration configuration,
ILogger<WafStandMiddleware> logger)
{
    private static readonly TimeSpan RegexTimeout = TimeSpan.FromMilliseconds(50);

    public static readonly IReadOnlyList<WafRule> Rules =
    [
        new("942100", "SQLi: UNION SELECT", new Regex(@"\bunion\b[\s\S]{0,20}\bselect\b",
RegexOptions.IgnoreCase | RegexOptions.CultureInvariant, RegexTimeout)),
        new("942130", "SQLi: тавтология ' OR 1=1", new Regex(@"['""]\s*(or|and)\s+['""]?\d+
['""]?\s*=\s*['""]?\d+", RegexOptions.IgnoreCase | RegexOptions.CultureInvariant,
RegexTimeout)),
        new("942350", "SQLi: DROP/TRUNCATE/DELETE после ;", new Regex(@";\s*(drop|truncate|
delete)\s+(table|from)\b", RegexOptions.IgnoreCase | RegexOptions.CultureInvariant,
RegexTimeout)),
        new("942160", "SQLi: слепая инъекция SLEEP()/WAITFOR", new Regex(@"\bsleep\s*\(\s*\d+\
s*\)|\bwaitfor\s+delay\b", RegexOptions.IgnoreCase | RegexOptions.CultureInvariant,
RegexTimeout)),
        new("941100", "XSS: тег <script>", new Regex(@"<\s*script\b", RegexOptions.IgnoreCase |
RegexOptions.CultureInvariant, RegexTimeout)),
        new("941120", "XSS: обработчик события / javascript:", new Regex(@"\bon(error|load|
click|mouseover)\s*=|javascript\s*:", RegexOptions.IgnoreCase | RegexOptions.CultureInvariant,
RegexTimeout)),
    ];

    public async Task InvokeAsync(HttpContext context)
    {
        var mode = configuration.GetValue("Waf:Mode", WafMode.On);
        long maxBody = configuration.GetValue("Waf:MaxBodyBytes", 65536L);
        context.Response.Headers["X-Waf-Mode"] = mode.ToString(); // smoke-тест проверяет режим
после деплоя

        if (mode == WafMode.Off)
        {
            await next(context);
```

```
            return;
        }

        // 1. Ограничение размера тела
        if (context.Request.ContentLength > maxBody)
        {
            if (await Violation(context, mode, "920350", $"Тело запроса
{context.Request.ContentLength} байт > лимита {maxBody}",
StatusCodes.Status413PayloadTooLarge))
            {
                return;
            }
        }

        // 2. Сбор проверяемых значений: путь, параметры строки запроса, значения полей JSON
        var values = new List<string> { Uri.UnescapeDataString(context.Request.Path.Value ??
"") };
        foreach (var (_, v) in context.Request.Query)
        {
            values.AddRange(v.Where(s => s is not null)!);
        }

        if (context.Request.ContentType?.Contains("json", StringComparison.OrdinalIgnoreCase)
== true && context.Request.ContentLength is > 0)
        {
            context.Request.EnableBuffering();
            try
            {
                using var doc = await JsonDocument.ParseAsync(context.Request.Body);
                CollectStrings(doc.RootElement, values);
            }
            catch (JsonException)
            {
                // некорректный JSON отклонит сам API (400) — WAF его не блокирует
            }
            finally
            {
                context.Request.Body.Position = 0; // тело остаётся доступным для API
            }
        }

        // 3. Проверка правил
        foreach (string value in values)
        {
            foreach (var rule in Rules)
            {
                bool hit;
                try
                {
                    hit = rule.Pattern.IsMatch(value);
                }
                catch (RegexMatchTimeoutException)
                {
                    hit = true; // защита от ReDoS: подозрительно «тяжёлое» значение
                }

                if (hit && await Violation(context, mode, rule.Id, rule.Description,
StatusCodes.Status403Forbidden))
                {
                    return;
                }
            }
        }

        await next(context);
    }

    /// <summary>Журналирует срабатывание; в режиме On блокирует запрос. Возвращает true, если
запрос заблокирован.</summary>
    private async Task<bool> Violation(HttpContext context, WafMode mode, string ruleId, string
description, int status)
    {
        string traceId = Activity.Current?.TraceId.ToString() ?? context.TraceIdentifier;
        if (mode == WafMode.DetectionOnly)
        {
            logger.LogWarning("WAF [DetectionOnly] правило {RuleId} «{Description}» сработало
на {Method} {Path} — запрос ПРОПУЩЕН, trace {TraceId}",
```

Листинг 6 – SmokeTests.cs — smoke-тесты № 1–3

```
    private readonly OrderServiceFactory _factory = new();
    private readonly HttpClient _client;

    public SmokeTests() => _client = _factory.CreateClient();

    [Fact(DisplayName = "1. GET /health возвращает 200 и версию релиза 2.0.0")]
    public async Task Health_Returns200()
    {
        using var response = await _client.GetAsync("/health");

        Assert.Equal(HttpStatusCode.OK, response.StatusCode);
        using var json = JsonDocument.Parse(await response.Content.ReadAsStringAsync());
        Assert.Equal("Healthy", json.RootElement.GetProperty("status").GetString());
        Assert.Equal("2.0.0", json.RootElement.GetProperty("version").GetString());
        Assert.Equal("On", response.Headers.GetValues("X-Waf-Mode").Single()); // WAF включён в
боевом режиме
    }

    [Fact(DisplayName = "2. DbContext.Database.CanConnect() — подключение к БД есть")]
    public void Database_CanConnect()
    {
        using var scope = _factory.Services.CreateScope();
        var db = scope.ServiceProvider.GetRequiredService<OrderDbContext>();

        Assert.True(db.Database.CanConnect());
    }

    [Fact(DisplayName = "3. WAF пропускает сложный JSON мобильного приложения (нет ложного
срабатывания)")]
    public async Task Waf_AllowsComplexMobileJson()
    {
        // Реальный по структуре заказ: вложенные объекты, опции, апострофы, кавычки,
        // «опасные» слова в обычном тексте — именно на таком ломаются неотлаженные правила WAF
        var order = new
        {
            customerId = "mobile-user-10",
            channel = "mobile",
            items = new object[]
            {
                new { sku = "PIZZA-30", name = "Пицца «4 сыра»", quantity = 2, price = 649.00m,
                      options = new Dictionary<string, string> { ["size"] = "30 см", ["crust"]
= "тонкое" } },
                new { sku = "DRINK-05", name = "Morse 0.5 l", quantity = 1, price = 120.00m,
options = (object?)null }
            },
            delivery = new { address = "ул. O'Connor, д. 5", entrance = "2", floor = "7",
timeSlot = "18:00-19:00" },
            comment = "Select the fastest courier please; order by phone, код домофона 15#.
Don't ring \"twice\""
        };

        using var response = await _client.PostAsJsonAsync("/api/v1/orders", order);

        Assert.Equal(HttpStatusCode.Created, response.StatusCode);
    }

    [Theory(DisplayName = "4. WAF блокирует атаки (SQL-инъекция, XSS) ответом 403")]
    [InlineData("/api/v1/orders/00000000-0000-0000-0000-000000000001?search=1%27%20OR
%20%271%27%3D%271", "942130")]
    [InlineData("/api/v1/orders/00000000-0000-0000-0000-000000000001?q=1%20UNION%20SELECT
%20password%20FROM%20users", "942100")]
    [InlineData("/api/v1/orders/00000000-0000-0000-0000-000000000001?name=%3Cscript
%3Ealert(1)%3C%2Fscript%3E", "941100")]
    public async Task Waf_BlocksAttacks(string url, string expectedRule)
    {
        using var response = await _client.GetAsync(url);

        Assert.Equal(HttpStatusCode.Forbidden, response.StatusCode);
        using var json = JsonDocument.Parse(await response.Content.ReadAsStringAsync());
        Assert.Equal(expectedRule, json.RootElement.GetProperty("ruleId").GetString());
    }

    [Fact(DisplayName = "5. WAF проверяет значения JSON-тела: XSS в комментарии заказа
блокируется")]
    public async Task Waf_BlocksXssInsideJsonBody()
```

4. Снимки экрана

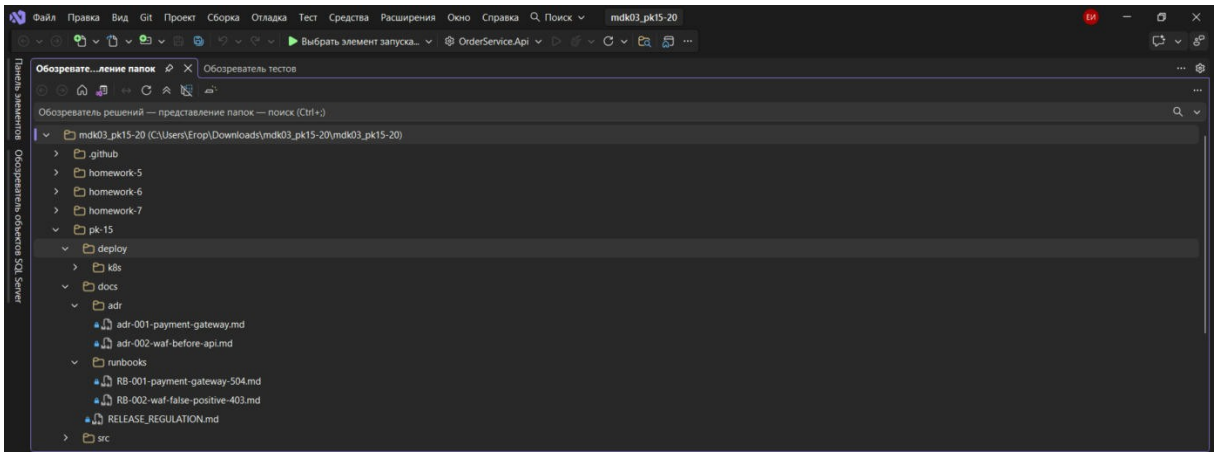


Рисунок 1 – Решение OrderService.sln в Visual Studio: проект, тесты и документация в папке docs

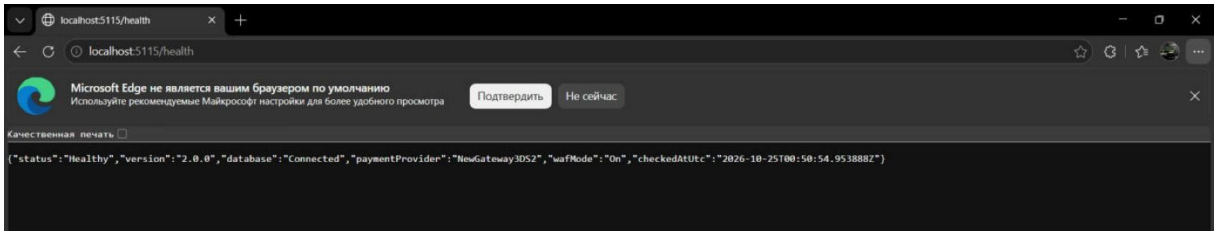


Рисунок 2 – Smoke-тест № 1: GET /health запущенного сервиса — 200, Healthy, версия 2.0.0, БД Connected, WAF On

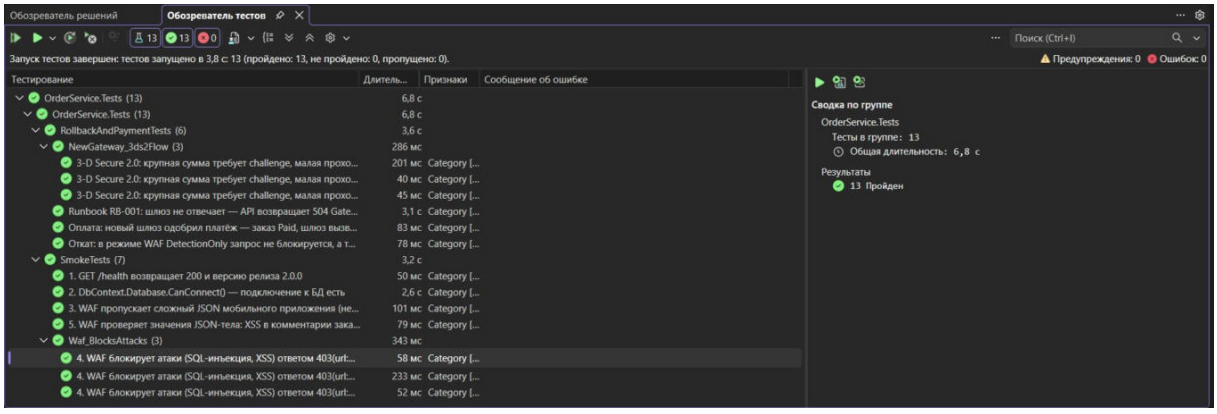


Рисунок 3 – Обозреватель тестов Visual Studio: все 13 тестов пройдены — smoke-тесты № 1–5 (в том числе WAF пропускает сложный JSON и блокирует SQL-инъекции/XSS), тест отката DetectionOnly, тесты оплаты с Moq

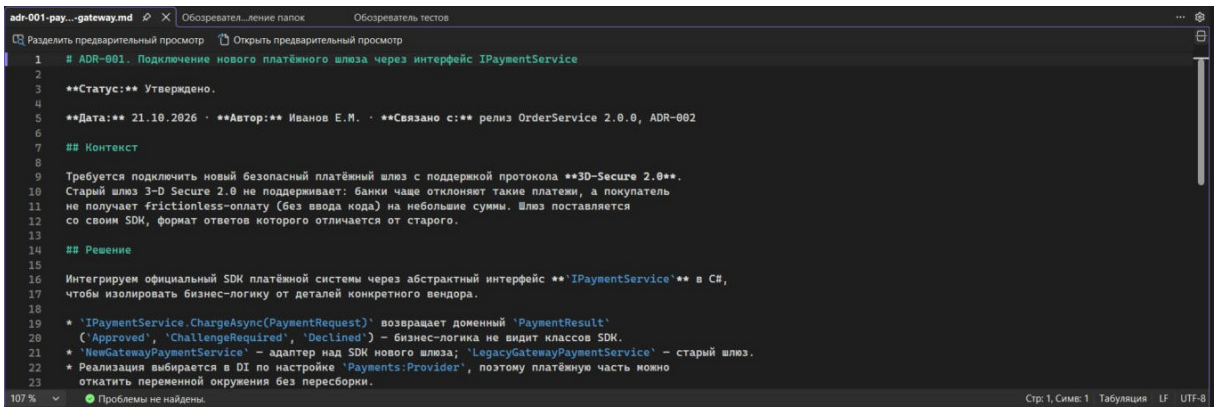


Рисунок 4 – Документ Б — docs/adr/adr-001-payment-gateway.md, открытый в Visual Studio

5. Ответ на вопрос

Какой из трёх элементов (план отката, Runbook или ADR) наиболее сложен для внедрения в команде и почему?

Наиболее сложным я считаю **план отката**. Runbook и ADR — это документы: их можно написать за час, и они начинают работать сразу. План отката работает только тогда, когда под него подготовлена система. Нужны версионированные образы и история ревизий в Kubernetes. Схема БД должна оставаться совместимой со старой версией (паттерн Expand and Contract). Старый шлюз остаётся в сборке и переключается настройкой, а у WAF должен быть режим DetectionOnly. Вдобавок команда заранее договаривается о числовом условии отката (5xx > 3 % за 3 минуты), иначе в 2 часа ночи начинаются споры «подождём ещё».

Самое трудное — проверить откат до аварии: его нужно регулярно отрабатывать на стенде, а это время, которое команды обычно тратят на новые функции. В моём кейсе сложность выше, потому что «откатить» WAF полностью нельзя — это снова откроет API для атак. Поэтому откат WAF — это перевод в DetectionOnly, а состояние после отката проверяется отдельным автотестом `DetectionOnly_DoesNotBlock`.

6. Вывод

В работе спроектирован релизный процесс OrderService 2.0.0 по CALS-подходу. Описаны окно обслуживания (среда 02:00–03:00 с оповещением

пользователей за 24 часа) и пять smoke-тестов. Составлен план отката с числовым условием активации и командами, укладывающимися в 30 секунд. Написаны Runbook на инцидент 504 Gateway Timeout и ADR, фиксирующий выбор интерфейса IPaymentService. По варианту 10 учтена специфика WAF перед API: внедрение в два этапа (DetectionOnly → On), проверка значений JSON вместо сырого текста, отдельный Runbook и откат на случай ложных срабатываний на мобильном трафике.

Все артефакты хранятся в репозитории рядом с кодом (Docs-as-Code). Регламент проверяется автотестами в Visual Studio: сложный JSON мобильного приложения проходит через WAF, атаки блокируются, а после отката запросы только журналируются.